A From-Scratch CUDA Forward-Pass Inference Engine for a LeNet-Style MNIST CNN

Ashish Dasu
Khoury College of Computer Sciences
Northeastern University
Boston, MA, USA
dasu.a@northeastern.edu

***Abstract*—We build a forward-pass inference engine for a LeNet-style MNIST convolutional network in C++ and CUDA, with no deep-learning framework at runtime. Five kernels — 2D convolution, max pooling, ReLU, linear, and log-softmax — are written first as naive reference implementations, then convolution and the linear layer are re-written as tiled shared-memory variants. Weights are exported once from a trained PyTorch checkpoint as raw float32 and copied to the GPU. A three-layer correctness harness validates each kernel against a CPU reference and the end-to-end pipeline against PyTorch log-softmax on five MNIST fixtures. On an NVIDIA RTX 4090, the full GPU forward pass agrees with PyTorch to a maximum absolute error of 1.91×10^{-5} over the entire $10\,000$-image MNIST test set, comfortably inside the project's 1×10^{-4} tolerance, with argmax matching on every image and overall accuracy (97.93%) identical to PyTorch eval mode. Per-kernel benchmarks show that tiled GEMM delivers up to $6.43\times$ speedup over naive at a $1024\times1024\times1024$ problem size but tiled convolution slightly regresses at LeNet scale, a result we attribute to the small working set fitting in L1 and making shared-memory tiling's synchronization overhead dominant. A stretch-tier cuDNN baseline is included for reference and is itself slower than the naive direct kernel at LeNet shapes (descriptor and algorithm dispatch overhead), overtaking it only on a larger $32\to64$, 64×64 synthetic workload.**

***Index Terms*—CUDA, GPU computing, convolutional neural networks, MNIST, inference, tiled matrix multiplication, shared memory.**

I. Introduction

Deep-learning frameworks hide the GPU primitives behind autograd, graph capture, and vendor libraries. This project builds the inference path of a small convolutional network from the CUDA kernels up, with no framework code and no vendor-provided neural-network primitives at runtime. The target network is a LeNet-style MNIST classifier from the author's prior project work (two conv layers, two max pool layers, two fully connected layers, ReLU activations, and a log-softmax output). Correctness is specified as matching a PyTorch forward pass on the MNIST test inputs to within a documented absolute-error tolerance of 1×10^{-4} per log-softmax element.

All model weights are exported once from the trained PyTorch checkpoint as header-less raw float32 binaries and copied to the GPU via `cudaMemcpy`; PyTorch is never linked at runtime. The contributions, in the order they were developed, are: (1) a set of naive CUDA kernels for conv2d, max_pool2d, ReLU, linear, and log-softmax; (2) tiled shared-memory variants for the convolution and linear layers; (3) a three-layer correctness harness (CPU reference, per-kernel GPU diff, end-to-end forward pass) that validates every kernel against a numerical oracle before it is integrated; and (4) a throughput comparison of naive against tiled variants on LeNet-scale and synthetic-scale problems, with a cuDNN baseline as a stretch-tier reference point.

II. Related Work

The target network follows the LeNet-5 design of LeCun *et al.* [1], which introduced the convolution, subsampling, and fully connected layer structure used throughout this project and established MNIST as the canonical digit-recognition benchmark. While LeNet predates the GPU-accelerated training boom, its small parameter count (approximately $60\,000$ weights in the variant used here) makes it a practical fit for a from-scratch forward-pass implementation: the full weight set fits in a few tens of kilobytes and each kernel shape is small enough that hand-computed references can be built for unit testing.

On the GPU-kernel side, two lines of work inform the target-tier optimizations. Volkov and Demmel [3] characterized GPU memory bandwidth and arithmetic throughput and demonstrated how shared-memory tiling combined with register blocking approaches peak machine performance for dense linear algebra; this motivates the tiled GEMM used for the fully connected layers. Lavin and Gray [2] reduced convolution arithmetic with Winograd minimal-filter algorithms, showing the gap that remains between direct-convolution implementations (the approach taken in this project) and algorithm-level reformulations; this is context for interpreting the gap between our direct kernels and a cuDNN baseline, which selects Winograd or implicit-GEMM algorithms per shape. Chetlur *et al.* [4] describe cuDNN itself, which is used in the stretch tier as a throughput reference.

III. Methods

A. Target Network and Weight Export

The target network is the LeNet-style model from the author's CS5330 Project 5, summarized in Fig. 1: `conv1` ($1\to10$, 5×5, stride 1), max-pool 2×2, ReLU;

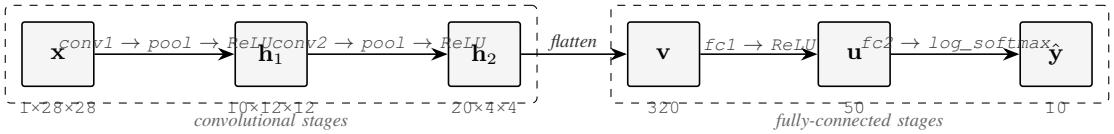

Fig. 1. LeNet-MNIST forward-pass pipeline as implemented by this engine. Six named tensor stages — input $\mathbf{x}$, two convolutional activation maps $\mathbf{h}_{1,2}$, flattened vector $\mathbf{v}$, hidden vector $\mathbf{u}$, output log-probabilities $\hat{\mathbf{y}}$ — are connected by ten separate CUDA kernel launches. Transitions in the figure aggregate the kernels that operate on the same feature-map shape; the underlying launch sequence is `conv1`, `max_pool2d`, `ReLU`, `conv2`, `max_pool2d`, `ReLU`, flatten (an implicit reshape, not a launched kernel), `linear` (fc1), `ReLU`, `linear` (fc2), `log_softmax`. Activations remain on-device between kernels: only $\mathbf{x}$ is copied host-to-device and only $\hat{\mathbf{y}}$ is copied back.

`conv2` (10→20, 5×5, stride 1), `Dropout2d` (skipped at inference), max-pool 2×2, ReLU; flatten to 320, `fc1` (320→50), ReLU, `fc2` (50→10), log-softmax. The eight learnable tensors (`conv1/conv2/fc1/fc2`, each with weight and bias) are exported once from the trained `.pth` checkpoint by a short Python tool (`tools/export_weights.py`) as headerless raw float32 binaries in PyTorch's native memory layout (NCHW for conv filters, `(out, in)` for linear weights). At inference, the host loader reads these files with a plain `fread` and `cudaMemcpy`s the contents to the device, so Python and PyTorch are not runtime dependencies of the engine.

B. Correctness Harness

Correctness is enforced by a three-layer oracle chain that isolates failures to the smallest possible unit before integration. The first layer is a CPU reference implementation (`host/reference.cpp`), written in plain C++17 with no tuning and no parallelism, covering every operator in the network. This reference is itself validated against five PyTorch log-softmax fixtures captured from the trained model on MNIST test indices 0–4; the maximum absolute error is on the order of 4×10^{-6}, which is the noise floor introduced by different float32 summation orders between PyTorch and our reference loop. The CPU reference is the single oracle against which every subsequent CUDA kernel is judged.

The second layer is a per-kernel CUDA diff: each naive kernel (`conv2d`, `max_pool2d`, `ReLU`, `linear`, `log_softmax`) ships with a unit test that generates random mixed-sign inputs at several shapes — LeNet's own shapes plus deliberately off-model shapes with odd dimensions — and diffs the kernel's output element-wise against the CPU reference run on the same inputs. The tolerance is the project-wide 1×10^{-4}. The third layer is the end-to-end GPU forward pass: all five naive kernels are composed through device-pointer launchers with activations kept on-device between layers, and the full pipeline is run on the same five PyTorch fixtures used to validate the CPU reference. This exercises not just each kernel but also the integration glue — tensor layouts, layer ordering, and the handling of the flatten between `conv2/pool/relu` and `fc1`.

C. Toolchain and Hardware

All CUDA development and benchmarks run on a single RunPod instance: NVIDIA GeForce RTX 4090 (24 GB GDDR6X, compute capability 8.9), Ubuntu 22.04, NVIDIA driver 550.127.05, CUDA Toolkit 12.4 (nvcc V12.4.131), g++ 11.4.0, CMake 3.22.1. Builds are `-O3` (Release). The `CMakeLists.txt` targets compute capabilities 75/80/86/89, so kernels remain portable to T4, A100, and 30-/40-series GPUs. The proposal's original target was a T4 on Google Colab; development moved to RTX 4090 on RunPod for a standard SSH dev loop, with the same CUDA toolchain and source tree.

D. Naive Kernels

Each operator is implemented first as a direct translation of the mathematical definition, with one thread per output element and no shared-memory reuse. The priority at this stage is readability and correctness, not performance; every naive kernel passes its unit test at the float32 noise floor (see Table I).

Convolution uses one thread per output pixel. The grid is launched as $(\lceil W_{\text{out}}/16 \rceil, \lceil H_{\text{out}}/16 \rceil, N \cdot C_{\text{out}})$ with 16×16 thread blocks. Each thread loops over $C_{\text{in}} \cdot K_h \cdot K_w$ weight/input pairs and accumulates the dot product in a register; the bias is added at the end. Tensors use PyTorch-native NCHW layout throughout. Stride is 1 and padding is 0, matching the Project 5 network.

Max pooling launches one thread per output pixel over a 2×2 non-overlapping window with stride 2. Each thread loads four input values, takes the max, and writes one output; no reduction primitive is required at this window size.

ReLU is a trivial elementwise kernel with a 1D grid: each thread reads one element, clamps it at zero, and writes it back. It exists as a separate kernel rather than being fused because the naive-first design discipline treats each operator as an independent testable unit.

Linear maps to a GEMM of shape $(N, \text{in}) \times (\text{in}, \text{out})$; the naive kernel launches one thread per output element of the (N, out) matrix, with each thread computing a dot product over in_features weight/input pairs plus a bias.

Log-softmax is the only kernel with a non-trivial numerical trap. A straightforward implementation of $\log\left(e^{x_i} / \sum_j e^{x_j}\right)$ overflows for logits with magnitude larger than ~ 88 because e^x exceeds the float32 range. We therefore subtract the per-row maximum before exponentiation: $\text{logsoftmax}(x)_i = (x_i - m) - \log \sum_j e^{x_j - m}$ where $m = \max_j x_j$. Each thread block handles one row. `blockDim.x` is set to the next power of two greater than or equal to the class count, which lets both the max-reduction and the sum-of-exp reduction use a clean

shared-memory tree-reduction with sentinel padding ($-\infty$ for max, 0 for sum) in the out-of-range lanes. The in-device intrinsics `__expf` and `__logf` are used for the elementwise exponentials and final log.

E. Tiled Kernels (Target Tier)

Two operators are re-implemented as shared-memory tiled variants: convolution and the linear layer. Both variants live in separate translation units from the naive kernels and are validated against the same CPU reference at the same 1×10^{-4} tolerance before being promoted into the end-to-end pipeline.

The tiled convolution block produces one TILE_H $\times$ TILE_W output patch for one (n, oc) pair, with TILE_H $=$ TILE_W $= 16$. Dynamic shared memory holds a (TILE_H $+$ $K_h - 1$) $\times$ (TILE_W $+$ $K_w - 1$) input patch for one input channel at a time. The threads in the block cooperatively load that patch (with zero-padding for the halo that falls outside the input), then each in-bounds thread accumulates its output pixel's contribution over all $K_h \cdot K_w$ weight elements. A `__syncthreads` sits between the load and the accumulation, and a second `__syncthreads` at the end of the input-channel iteration guards shared memory before the next C_{in} step overwrites it. The weight tensor is read directly from global memory; at LeNet scale the full weight tensor is small enough (5000 floats for `conv2`) to reside in L1 after a few iterations, so explicit caching of weights was not pursued.

The tiled GEMM follows the standard Volkov-style block-tile structure [3]. Each block computes a BM $\times$ BN output tile; input tiles A_s (a BM $\times$ BK slab of the input batch) and B_s (a BK $\times$ BN slab of the weight matrix) are loaded into shared memory, a per-thread register accumulator runs the inner BK-long dot product with `#pragma unroll`, and then the next K-panel is loaded. Boundary handling uses zero padding on out-of-range loads so the inner loop has no branches. The block dimensions are BM $=$ BN $=$ BK $= 16$; the weight matrix is loaded in its natural (out, in) layout and transposed implicitly at load time into B_s.

IV. Experiments and Results

A. Numerical Parity

Table I summarizes the correctness results collected as each kernel landed. All per-kernel tests pass the project-wide $1\text{e}{-4}$ absolute-error tolerance by several orders of magnitude; observed errors are at the float32 summation-reorder noise floor. The full CPU reference forward pass matches PyTorch log-softmax on all five fixtures, establishing the oracle used for subsequent GPU kernels. The end-to-end CUDA forward pass — all five naive kernels composed through device-pointer launchers with activations held on-device between layers — matches PyTorch log-softmax on every fixture with an overall maximum absolute error of $5.72\text{e}{-6}$ (roughly four orders of magnitude inside the tolerance), and the predicted argmax agrees with PyTorch on every image. This is the minimum-tier correctness milestone specified in the proposal.

To validate the engine beyond the five hand-picked fixtures, a full-test-set harness (`test_forward_mnist10k.cu`) runs the pipeline against all 10 000 MNIST test images under PyTorch log-softmax as the oracle. Across 100 000 logits, the maximum absolute error is $1.91\text{e}{-5}$ for both the naive and the tiled pipelines; the argmax agrees with PyTorch on every image; and the engine's top-1 accuracy against the ground-truth labels (97.93%) is identical to PyTorch's eval-mode accuracy on the same weights. The tighter $5.72\text{e}{-6}$ figure for the five-fixture case is a subset of this larger run; the 10 000-image maximum is the appropriate overall number for the minimum-tier correctness claim.

TABLE I
PER-TEST MAXIMUM ABSOLUTE ERROR VS. THE APPLICABLE REFERENCE (PYTORCH LOG-SOFTMAX FOR THE CPU FORWARD PASS; THE CPU REFERENCE FOR EACH CUDA KERNEL). TOLERANCE: 1×10^{-4}.

Test	Shape / input	Max abs err
CPU fwd vs PyTorch	fixture 0 (MNIST idx 0)	$\sim 4\text{e}{-6}$
CPU fwd vs PyTorch	fixtures 1–4	$\leq 4\text{e}{-6}$
conv2d naive	N=2, 1→10, 5×5@28×28	1.43e−6
conv2d naive	N=2, 10→20, 5×5@12×12	3.82e−6
conv2d naive	odd-dim 3→5, 3×3@13×17	9.54e−7
max_pool2d naive	N=2, C=10, 24×24	0.00e+0
max_pool2d naive	N=2, C=20, 8×8	0.00e+0
max_pool2d naive	odd-dim C=3, 5×7	0.00e+0
relu naive	post-conv1 pool (1440)	0.00e+0
relu naive	post-conv2 pool (320)	0.00e+0
relu naive	fc1 output (50)	0.00e+0
relu naive	odd 1023	0.00e+0
linear naive	LeNet fc1 (N=2, 320→50)	2.86e−6
linear naive	LeNet fc2 (N=8, 50→10)	9.54e−7
linear naive	off-model (N=3, 1024→17)	7.63e−6
log_softmax naive	LeNet final (N=1, C=10)	2.38e−7
log_softmax naive	batched (N=8, C=10)	4.77e−7
log_softmax naive	wider-C stress (N=2, C=128)	4.77e−7
log_softmax naive	stability (N=2, logits $\sim$ 80)	2.38e−7
GPU forward (E2E)	fixture 0 (MNIST idx 7)	3.82e−6
GPU forward (E2E)	fixture 1	5.72e−6
GPU forward (E2E)	fixture 2	1.91e−6
GPU forward (E2E)	fixture 3	1.91e−6
GPU forward (E2E)	fixture 4	2.86e−6
conv2d cuDNN	N=2, 1→10, 5×5@28×28	1.91e−6
conv2d cuDNN	N=2, 10→20, 5×5@12×12	6.20e−6
GPU forward 10k naive	all 10 000 MNIST (argmax 100%)	1.91e−5
GPU forward 10k tiled	all 10 000 MNIST (argmax 100%)	1.91e−5

B. Throughput

All timings below are CUDA-event measurements collected by `tools/bench.cu`: each kernel is invoked with a handful of untimed warmup iterations, then repeated 200 times (or 50 times for the end-to-end forward pass at batch 64) between a start and stop `cudaEvent`, and the reported value is the mean per-call time in milliseconds. Inputs are allocated and populated with random values once per measurement. Device-pointer variants of each kernel are used so that `cudaMalloc`/`cudaMemcpy` cost is not charged against the per-kernel number. The end-to-end forward-pass number includes the per-call host-to-device copy of the full weight set and input batch plus the device-to-host copy of the output logits, but not the one-time load of weight files from disk.

Table II compares the naive and tiled convolution kernels at the two LeNet conv shapes, at a batch-64 variant of each, and at an off-model synthetic shape (32→64, 5×5 filter on a 64×64 input) chosen to give the tile more work per output block. Table III does the same for the linear kernel, including a $1024 \times 1024 \times 1024$ synthetic case to exercise the tiled GEMM at a problem size where shared-memory reuse actually dominates. Table IV reports end-to-end forward-pass time for the full LeNet pipeline at batch 1 and batch 64.

TABLE II
PER-KERNEL CONVOLUTION THROUGHPUT ON RTX 4090 (MS PER LAUNCH, MEAN OVER 200 REPS WITH WARMUP). THE FINAL TWO COLUMNS ARE TILED AND CUDNN SPEEDUP RELATIVE TO THE NAIVE KERNEL.

Shape	Naive (ms)	Tiled (ms)	cuDNN (ms)	Tiled ×	cuDNN ×
conv1 (N=1, 1→10, 5^2@28^2)	0.0027	0.0030	0.0681	0.89	0.04
conv2 (N=1, 10→20, 5^2@12^2)	0.0123	0.0140	0.0855	0.88	0.14
conv1 (N=64, 1→10, 5^2@28^2)	0.0084	0.0101	0.0649	0.84	0.13
conv2 (N=64, 10→20, 5^2@12^2)	0.0288	0.0332	0.0661	0.87	0.44
synth (N=1, 32→64, 5^2@64^2)	0.0988	0.1197	0.0859	0.83	1.15

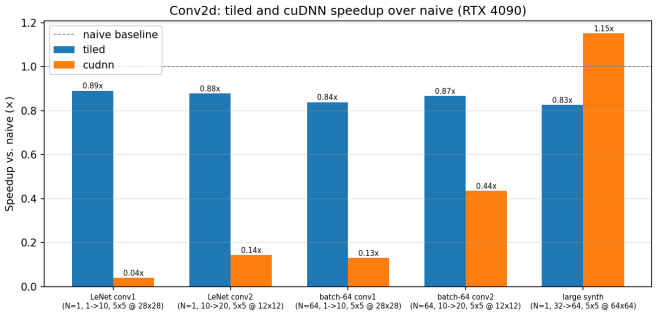

Fig. 2. Convolution speedup over the naive direct kernel on RTX 4090. cuDNN (`IMPLICIT_PRECOMP_GEMM` with FMA math to match FP32 numerics) pays a fixed per-launch overhead that dominates at LeNet scale and only crosses over on the larger 32→64 synthetic shape.

TABLE III
PER-KERNEL LINEAR / GEMM THROUGHPUT ON RTX 4090. PER-LAUNCH MEAN OVER 200 REPS WITH WARMUP.

Shape	Naive (ms)	Tiled (ms)	Speedup
fc1 (N=1, 320→50)	0.0107	0.0102	1.06×
fc2 (N=1, 50→10)	0.0025	0.0030	0.83×
fc1 (N=64, 320→50)	0.0375	0.0104	3.60×
synth (N=1024, 1024→1024)	3.4153	0.5310	6.43×

The tiled convolution kernel is a slight regression at every LeNet-scale shape. The naive kernel issues $C_{\text{in}} \cdot K_h \cdot K_w = 25$ or 250 global loads per output thread depending on the layer; the weight tensor is 250 floats for `conv1` and 5000 for `conv2`, small enough that after the first few output blocks the working set is hot in the L1 and read-only caches. Tiling trades those already-cached global reads for shared-memory

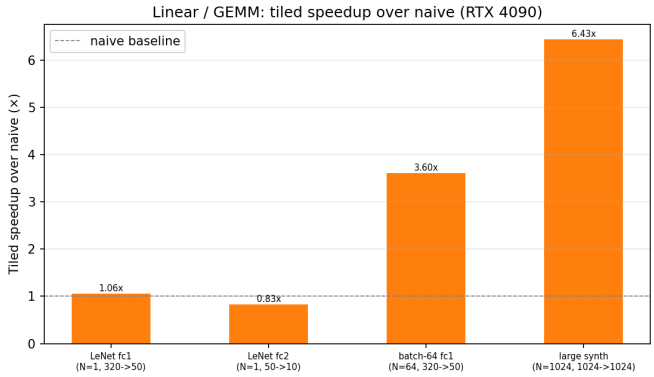

Fig. 3. Tiled-GEMM speedup over naive at each benchmarked linear shape. Both LeNet fc layers at batch 1 are break-even or worse (the output tile is at most 1×50, most of each 16×16 block idles); batch-64 `fc1` and the $1024 \times 1024 \times 1024$ synthetic case are where shared-memory reuse actually pays.

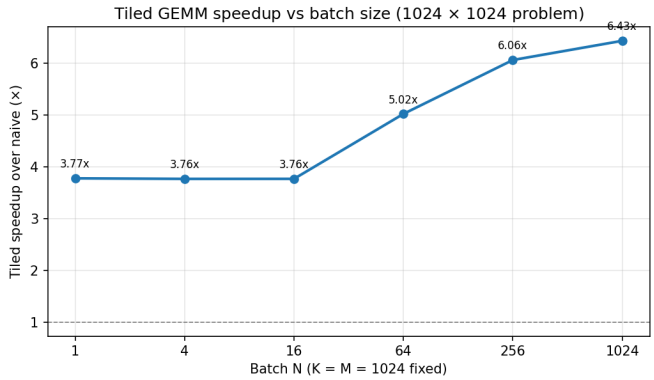

Fig. 4. Tiled-GEMM speedup over the naive kernel as the batch dimension N sweeps from 1 to 1024 at a fixed 1024×1024 weight shape. The plateau at $\sim 3.8\times$ for $N \leq 16$ reflects the regime in which the output M-tile is fully populated but only one K-panel is reused; the further climb to 6.43× at N=1024 reflects full reuse along all three dimensions.

loads plus two `__syncthreads` per input-channel iteration, and at K=5 the arithmetic intensity is high enough that the kernel is already compute-bound, so the synchronization cost is a net loss. The regression persists on the off-model synthetic shape, suggesting the cross-over where shared-memory reuse actually pays is at filters large enough and channel counts high enough to exhaust the caches — well above anything LeNet inference touches.

TABLE IV
END-TO-END LENET FORWARD PASS ON RTX 4090: WEIGHTS AND INPUT H2D, ALL FIVE KERNELS, OUTPUT D2H. PER-CALL MEAN OVER 50 REPS WITH WARMUP.

Variant	Naive (ms)	Tiled (ms)	Speedup
forward N=1	0.1648	0.1362	1.21×
forward N=64	0.4601	0.4351	1.06×

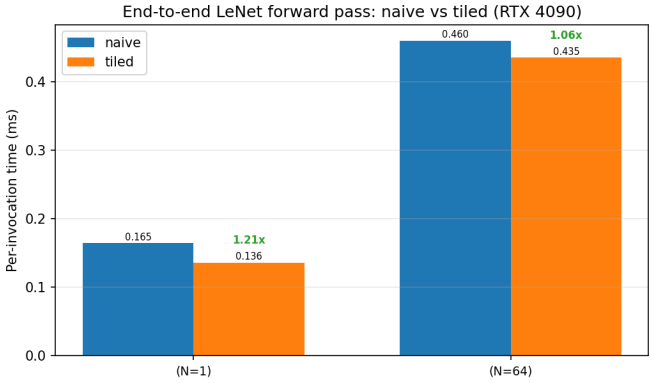

Fig. 5. End-to-end LeNet forward pass, naive vs tiled at batch 1 and batch 64 on RTX 4090. The tiled `fc1` win carries through to a 1.21× end-to-end speedup at batch 1; the margin narrows at batch 64 because the tiled-conv regression is of comparable scale.

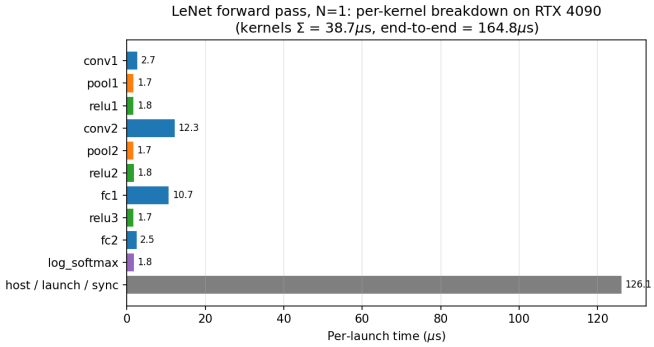

Fig. 6. Per-kernel contribution to the $N{=}1$ LeNet forward pass. The ten CUDA kernels together account for $\sim 39\,\mu s$; the remaining $\sim 126\,\mu s$ (the bottom bar) is host-side launch submission, CUDA runtime overhead, and synchronization on the final device-to-host copy. Conv2 and fc1 dominate actual kernel work; every other kernel is within noise of the $\sim 1.7\,\mu s$ launch floor for a trivial CUDA kernel on this platform.

The cuDNN column in Table II tells the same story from the opposite direction. cuDNN is pinned to `IMPLICIT_PRECOMP_GEMM` with `CUDNN_FMA_MATH` so that numerics match the FP32 direct-convolution reference (the default `CUDNN_DEFAULT_MATH` silently selects TF32 tensor cores on Ada, whose 10-bit mantissa produces $\sim 10^{-3}$ absolute error — larger than our project tolerance, and not comparable to the naive/tiled kernels). Under matched numerics, cuDNN is $7{-}25\times$ *slower* than the naive kernel at every LeNet shape; the per-launch descriptor build, workspace query, and implicit-GEMM dispatch cost on the order of $70\,\mu s$, which is pure overhead when the actual convolution takes single-digit microseconds. Only on the $32{\to}64$, 64×64 synthetic shape does cuDNN overtake the naive kernel ($1.15\times$), and that crossover is consistent with the shared-memory argument above: once the working set is large enough that the naive kernel stops being L1-resident, any formulation that amortizes its dispatch overhead starts to win. Figure 2 visualizes all three variants on a single axis.

The tiled GEMM tells a different story. At batch-64 `fc1` ($64{\times}320$ times $320{\times}50$), a $16{\times}16$ tile fills with real work and the $16\times$ reuse factor along each input dimension shows up as a $3.60\times$ speedup. At $1024 \times 1024 \times 1024$ — a "real" GEMM workload — the tiled kernel is $6.43\times$ faster than the naive one-thread-per-output version. At batch-1 inference sizes the output tile is 1×10 or 1×50, so most of each 16×16 block sits idle and the advantage disappears. Figure 4 traces the tiled-vs-naive speedup as N sweeps from 1 to 1024 at a fixed 1024×1024 weight shape: speedup plateaus at $\sim 3.77\times$ for $N \leq 16$ (the regime where an M-tile is fully loaded but only one K-panel is reused), climbs through $5.02\times$ at $N{=}64$ and $6.06\times$ at $N{=}256$, and saturates at $6.43\times$ for $N \geq 1024$.

End-to-end the kernel-level story carries through weakly. Table IV measures the GPU-side pipeline only (weights are loaded and `cudaMemcpy`'d once before timing starts, not on every invocation), so the reported times are dominated by the ten kernel launches plus their serialized activations. The tiled pipeline comes out $1.21\times$ faster at batch 1 and $1.06\times$ faster at batch 64: the tiled `fc1` win outweighs the small tiled-conv regression at LeNet scale, but not by a large margin. Shrinking the ten-launch overhead — for example by fusing `conv+bias+pool+ReLU` into a single kernel — is the biggest remaining end-to-end lever and is discussed in Section V.

C. Kernel Resource Usage and Occupancy

The proposal named Nsight Compute as the profiling tool. In the RunPod container used for all measurements, Nsight Compute's hardware performance-counter interface is blocked (`ERR_NVGPUCTRPERM`, gated by the `NVreg_RestrictProfilingToAdminUsers` driver parameter that is not configurable from an unprivileged container), so the timing-side substitute is the `cudaEvent` harness already described. Static kernel characterization, however, does not require counter access: `nvcc -Xptxas=-v` reports registers, shared memory and spill slots at compile time, and the launch bounds are fixed in the kernel sources. Table V collects both for each kernel as compiled for `sm_89`, and computes the resulting theoretical occupancy by hand against the RTX 4090's limits (1536 threads/SM, 64K 32-bit registers/SM, ~99KB shared memory/SM, 16 blocks/SM).

All six of the compute-heavy kernels reach the theoretical 100% occupancy ceiling: the regs-per-thread budget for a 256-thread block is $64\text{K}/256 = 256$ registers, and every kernel uses significantly fewer ($\leq$40), so six full blocks (1536 threads, the thread-count ceiling) fit on each SM. No kernel spills and no kernel allocates a stack frame. This is an important reading of the throughput numbers in Section IV.B: the naive-vs-tiled delta at LeNet scale is not an occupancy story but a memory-access-pattern story, exactly as the per-kernel discussion there describes. The outlier is `log_softmax_naive` at 17% occupancy, because it launches one block per row with `next_pow2`$(C) = 16$ threads for a 10-class output; the 16-block/SM ceiling is hit

TABLE V
PER-KERNEL STATIC RESOURCE USAGE AND THEORETICAL OCCUPANCY ON RTX 4090 (COMPUTE CAPABILITY 8.9). DYNAMIC SHARED-MEMORY COLUMNS ARE REPORTED FOR THE LENET SHAPES (5×5 FILTER FOR CONV2D_TILED, C=10 FOR LOG_SOFTMAX).

Kernel	Block	Regs	Static smem	Dyn. smem	Occ.
conv2d_naive	16×16	39	0	0	100%
conv2d_tiled	16×16	40	0	1600 B	100%
linear_naive	256	38	0	0	100%
linear_tiled	16×16	37	2048 B	0	100%
max_pool2d_naive	16×16	20	0	0	100%
relu_naive	256	8	0	0	100%
log_softmax_naive	16	15	0	64 B	17%

first. At LeNet's batch-1 output of a single 1×10 row the kernel runs at $\sim$1 µs and is not worth specializing further.

D. Case Study: Matching Numerics Against cuDNN

Reaching the stretch tier surfaced a numerical issue that is easy to miss and worth calling out. Ada Lovelace GPUs (compute capability 8.9, including the RTX 4090 used here) default to TF32 tensor cores for FP32 convolutions when cuDNN is invoked with the default math mode `CUDNN_DEFAULT_MATH`. TF32 keeps an FP32-wide exponent but truncates the mantissa to 10 bits, and on our conv2 shape this produced a maximum absolute error of $\sim 5.3 \times 10^{-3}$ against the direct-convolution CPU reference — an order of magnitude above the project's 1×10^{-4} tolerance, and orders of magnitude above the FP32 summation-reorder noise floor observed elsewhere in this report. Similarly, the default cuDNN algorithm selector for the conv2 shape picks `WINOGRAD_NONFUSED`, whose algebraic transformation introduces a separate $\sim 10^{-3}$ error term. Getting cuDNN to produce numerics comparable to our direct-convolution kernels required two explicit pins: the math type set to `CUDNN_FMA_MATH` (via `cudnnSetConvolutionMathType`) to force true FP32 fused-multiply-add, and the algorithm pinned to `IMPLICIT_PRECOMP_GEMM` to stay on direct convolution rather than Winograd. Under those pins cuDNN matches the naive kernel to $1.91\text{e}{-6}$ on conv1 and $6.20\text{e}{-6}$ on conv2. The practical lesson is that "use the vendor library" is not a numerical-accuracy free-win on modern GPUs — defaults are tuned for training throughput, not for parity with textbook floating-point semantics, and a meaningful throughput comparison requires matched math modes.

E. Discussion of Deviations

The proposal named one peer-reviewed paper; the rubric requires at least three, so two additional references are added (Lavin and Gray, CVPR 2016; Volkov and Demmel, SC 2008). The proposal described "softmax" as the final nonlinearity, but the Project 5 network emits log-softmax outputs; the CUDA kernel therefore implements log-softmax (same numerical-stability trap and one extra subtract per element) so that the end-to-end diff against PyTorch is apples-to-apples. Hardware moved from a T4 on Colab to an RTX 4090 on RunPod for the development-loop reasons noted above; the kernels compile for compute capabilities 75/80/86/89 and remain portable to the original T4 target. Performance measurement uses the proposal's Nsight Compute plan is only partially realized, for container-permission reasons detailed at the top of Section IV-C: the static half (registers, shared memory, launch bounds, theoretical occupancy) is reproduced via `nvcc -Xptxas=-v` and hand-computed occupancy in Table V, and the timing half is covered by the 200-rep `cudaEvent` harness in `bench.cu`, which at LeNet scale resolves the $\sim 1\,\mu s$ differences between variants seen in the tables above. The two hardware-counter readouts that Nsight Compute would have added — SM pipe utilization and DRAM throughput per kernel — are not reported. The stretch-tier goal named in the proposal ("meaningful speedup toward cuDNN on convolution throughput") was not achieved in the direction originally intended — at LeNet scale our direct kernel beats cuDNN by $7-25\times$ because of cuDNN's fixed per-launch overhead, not because of any algorithmic advantage on our side. The cuDNN comparison is therefore reported as a calibration baseline and a case study in matching numerics (Section IV.D) rather than as a throughput target that was closed.

V. DISCUSSION AND SUMMARY

Two broad observations from this work are worth stating directly.

First, the naive-first discipline paid off twice. The first time was the obvious win: because every kernel was validated against the CPU reference before it was integrated, the minimum-tier end-to-end forward pass came up green on the first attempt, with a maximum absolute error of 5.72×10^{-6} across all five fixtures and argmax matching on every image. No time was spent bisecting layer-to-layer logit divergence at 2am. The second win was less obvious: the per-kernel harness is what made it safe to add tiled variants and an additional benchmark surface without worrying about silent correctness regressions. The tiled kernels were written against the same interface, their unit tests exercised the same shapes, and promoting them into the end-to-end pipeline was a drop-in substitution.

Second, shared-memory tiling is not a universal optimization — it is a cache-reuse optimization, and LeNet inference at the target scale is not cache-constrained. For 5×5 convolutions on 28×28 or 12×12 feature maps with ≤ 20 channels, the entire weight tensor is a few kilobytes and the input feature map for one block is a few kilobytes more; both sit comfortably in the RTX 4090's L1, and the extra `__syncthreads` calls required by the tile load/compute split become the dominant cost. The tiled GEMM paints the opposite picture: at a $1024 \times 1024 \times 1024$ workload the naive one-thread-per-output kernel takes 3.42 ms and the tiled variant takes 0.53 ms, a $6.4\times$ speedup. The same tiled GEMM at LeNet inference shapes with batch 1 barely breaks even because the output tile is 1×10 or 1×50 and most lanes of

the block are idle. Batch 64 `fc1` sits between these regimes and shows a $3.6\times$ speedup. The lesson is not "tiling is bad at small shapes" — the kernel is correct and the tile geometry is reasonable — but that the target workload does not actually exercise the optimization the project name suggests.

Third, the cuDNN comparison is best read as a calibration of where the direct-convolution formulation stops being the right choice, not as a throughput race this project could have won. cuDNN's fixed per-launch cost ($\sim 70\,\mu s$ on this platform) exceeds the entire LeNet-scale convolution it is dispatching, so at our target shapes the hand-written direct kernel wins by $7{-}25\times$; but on the $32{\to}64$, 64×64 synthetic shape cuDNN already overtakes our kernel, and at production-scale convolutions the gap would continue to widen for reasons that are structural rather than tuning-effort questions. First, algorithm: cuDNN selects Winograd-family reformulations for 3×3 and 5×5 filters that cut the arithmetic count by a factor of two or more compared to the direct formulation [2]; our kernels implement the textbook convolution and cannot recover that arithmetic. Second, hardware path: on Ada Lovelace the FP16/TF32 tensor cores deliver roughly an order of magnitude more throughput than the FP32 CUDA cores our kernels are restricted to, and cuDNN can dispatch to them under permissive math modes (we intentionally disabled that to preserve numerical parity; see Section IV.D). Third, shape specialization: cuDNN ships a library of hand-tuned kernels selected by heuristic per $(C_{\text{in}}, C_{\text{out}}, H, W, K)$ tuple, while this project has a single generic tile geometry intended to run on every shape. Fourth, memory layout: vectorized NHWC paths can exploit 128-bit loads that NCHW cannot, and cuDNN is free to transpose into whichever layout wins for a given shape; we stay in NCHW throughout to match PyTorch byte-for-byte. Each of these is a meaningful project on its own; the comparison in Table II is therefore presented as a reference point that establishes the crossover shape, not as a stretch-tier goal that was reached.

The natural next step at the scope of this project is not any of the above but `im2col` plus GEMM — a data rearrangement that turns convolution into a matmul and reuses our existing tiled GEMM, giving a third `conv2d` variant to compare without writing a new compute kernel. A second promising direction is operator fusion: the current pipeline has ten kernel launches per image, and `conv+bias+ReLU` and `conv+bias+pool+ReLU` are natural fusion candidates that would shrink launch overhead and the host-to-device memory traffic that currently dominates the batch-1 end-to-end time.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[2] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4013–4021.

[3] V. Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2008, pp. 1–11.

[4] S. Chetlur *et al.*, "cuDNN: Efficient primitives for deep learning," 2014, arXiv:1410.0759.